[8] As long as it satisfies the requirements for a valid metric: being always non-negative, being null only between a point and itself, being symmetrical, and abiding by the triangular inequality.

[9] Remember: one of the golden rules of clean code is breaking up long, complex methods into smaller ones, so that each method is focused on one goal only.

[10] Assuming a point's centroid is the point itself. We will also assume points have radius equal to 0.

[11] Disclaimer: a function that returns a value *and* has a side effect is far from ideal, not the cleanest design. Using indirect sorting would be a better solution – here, we used the simplest solution because of limited space, but be advised.

[12] For Ss-Trees we partition the ordered list of points by selection the split index for the list, and then each point on the left of the index goes in one partition, and each point on the right on the other partition.

[13] We could reuse `search` as the first call in `delete` (and `searchClosestLeaf` in `insert`) if we store a pointer to its parent in each node, so that we can climb up the tree when needed.

[14] Assuming you implement this method as a private method: assertions should never be used to check input, because assertions can be (and often are, in production) disabled. Checking arguments in private methods is not ideal, and must be avoided when they are forwarded from user input: ideally, you would only use assertions on invariants.

[15] This can easily be extended to n-nearest neighbor by considering the distance of the n-th NN instead.

[16] Amato, Giuseppe. Approximate similarity search in metric spaces. Diss. Technical University of Dortmund, Germany, 2002.

[17] This section includes advanced material focused on theory.

[18] White, David A., and Ramesh Jain. "Similarity indexing with the SS-tree." Proceedings of the Twelfth International Conference on Data Engineering. IEEE, 1996.

[19] Cleary, John Gerald. "Analysis of an algorithm for finding nearest neighbors in Euclidean space." ACM Transactions on Mathematical Software (TOMS) 5.2 (1979): 183-192.

[20] Fischer, Kaspar, Bernd Gärtner, and Martin Kutz. "Fast smallest-enclosing-ball computation in high dimensions." European Symposium on Algorithms. Springer, Berlin, Heidelberg, 2003.

[21] The SS+-Tree: An Improved Index Structure for Similarity Searches in a High-Dimensional Feature Space.

[22] To be thorough, we should also consider that computing each distance requires `O(d)` steps, so the running time, if `d` can vary, becomes `O(djkn)`. While for Ss-trees we have seen how it becomes important to keep in mind how algorithms perform when the dimension of the space grows, a linear dependency is definitely good news, and we can afford to omit it (as it is done by convention, considering the dimension fixed) without distort our result.